locnht3@gmail.com

Object-Oriented Programming with Java

I. Variable & Data type **2**
1. Variable 2
2. Data type 2
II. Function **4**
III. Object & Class **5**
1. Object 5
2. Class 5
3. Constructor - Phễu 6
4. Access Specifier / Access Modifier 6
5. Object lưu trữ trong RAM như thế nào? 7
IV. Static **8**
V. Input & Try-catch **8**
VI. Array **9**
1. Primitive array 9
2. Object array 10
VII. Inheritance **11**
1. Định nghĩa 11
2. Class Con, Class Cha lưu trữ trong RAM như thế nào? 13
3. Abstract class 15
4. Các bước thực hiện Inheritance 16
5. Anonymous Class 17
VIII. Interface **18**
IX. OOP Concepts / OOP Principles **21**
1. Abstraction - trừu tượng hóa mọi vật 21
2. Encapsulation - cô lập đóng gói trọn vẹn đúng thứ cần làm 22
3. Inheritance - tái sử dụng, có xài luôn 23
4. Polymorphism - đa hình/đa ánh xạ - từ một ra nhiều 24

X. Collections Framework **24**
1. Tủ loại 1: Mảng [Số đồ để vô] 24
2. Tủ loại 2 25
XI. Wrapper Class **26**
1. Tổng quan 26
2. Pool 28

I. Variable & Data type

1. Variable

- Biến là một vùng ram được đặt tên/định danh/identifier, chiếm một số byte nhất định tùy hình thái dữ liệu mà nó sẽ chứa, dùng để lưu trữ một value nào đó.
- Biến là cách đặt tên cho một đại lượng, một giá trị.
  Vd: int yob = 2003;
  double vat = 0.1;

2. Data type

- Hình thái dữ liệu gồm: primitive, object.
- Primitive data type gồm:

Kiểu dữ liệu	Số bit	Giá trị nhỏ nhất	Giá trị lớn nhất
byte	8	-128 (-2^7)	127 (2^7-1)
short	16	-32,768 (-2^15)	32,767 (2^15 -1)
int	32	– 2,147,483,648 (-2^31)	2,147,483,647 (2^31 -1)
long	64	-9,223,372,036,854,775,808 (-2^63)	9,223,372,036,854,775,807 (2^63 -1)
float	32	-3.4028235 x 10^38	3.4028235 x 10^38
double	64	-1.7976931348623157 x 10^308	1.7976931348623157 x 10^308
boolean	false	true	
char	16	'\u0000' (0)	'\uffff' (65,535).

+ Mọi giá trị xuất hiện trong code, số nào đó, chữ nào đó xuất hiện trong code (3.14, 10, "age" …) được gọi là literal value.
+ Mọi literal value của số nguyên xuất hiện trong code được xem là kiểu int - 4 bytes.
  Vd: int year = 2026;
  long money = 2_200_000_000L;
  int status = 0xFA; // hệ hexa, 250 (decimal)
  int phone = 070; // hệ octal, 56 (decimal)

+ Mọi literal value của số thực được xem là kiểu double - 8 bytes.
  Vd: double pi = 3.14;
  float vat = 0.1F;

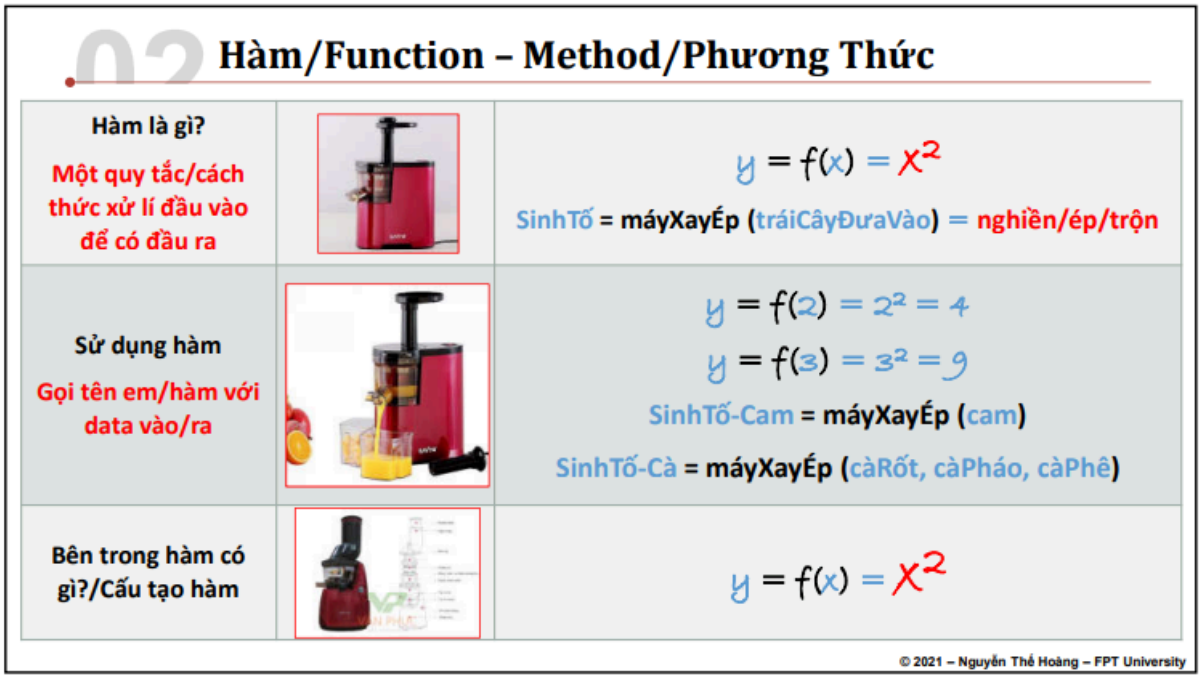
+ Java hỗ trợ kiểu char 2 bytes, dùng dấu '' để biểu thị là một ký tự đơn.
  Vd: char unit = '$';

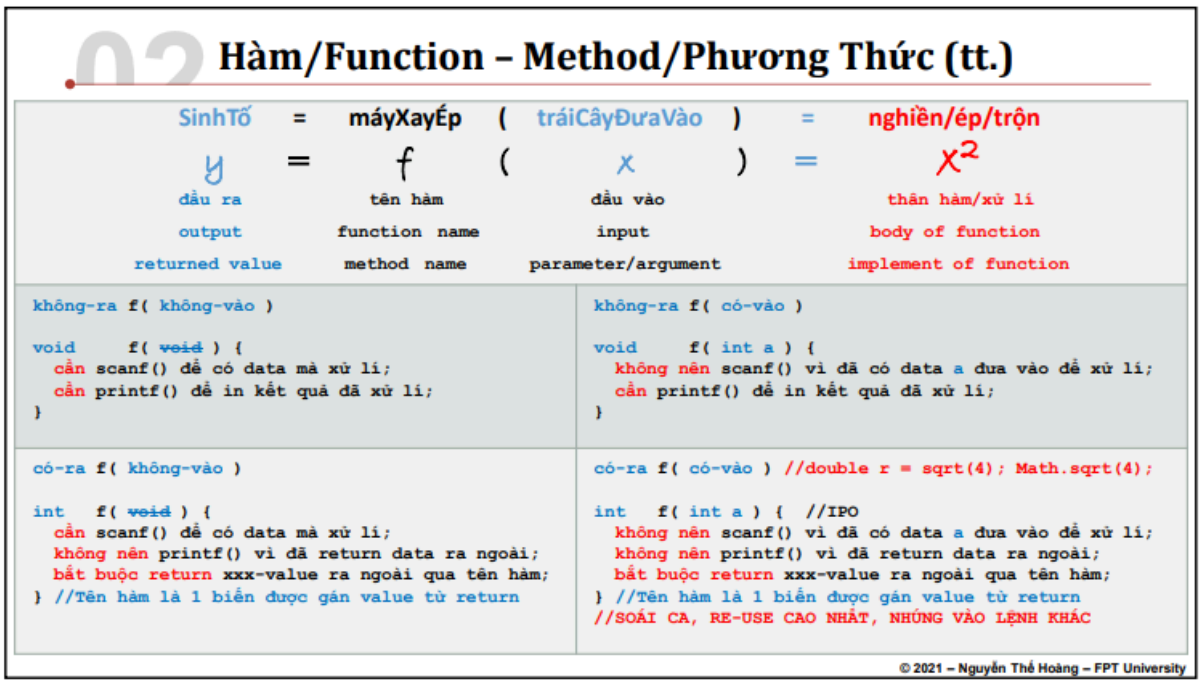
+ Trong một biểu thức so sánh, một mệnh đề, một câu phát biểu sẽ rơi vào 2 trạng thái: đúng - sai.
  - C quy ước: đúng là 1, sai là 0 => kiểu int.
  - Java quy ước: đúng là true, sai là false => kiểu boolean.

Vd: boolean marriedStatus = false;

- Java cung cấp một data type hoàn toàn mới, dùng để lưu trữ chuỗi - String. String dùng để lưu trữ, tên gọi cho những thứ phức tạp, một đống các ký tự ghép lại với nhau - kiểu object.
  Vd: String name = “Ngô Huỳnh Tấn Lộc”;

II. Function





- Hiện tượng trong 1 class bất kỳ mà có 2 hàm trùng tên nhau, nhưng khác phần tham số đầu vào, mà phải khác trên data type, không quan tâm tên biến, hiện tượng này được gọi là OVERLOAD - QUÁ TẢI HÀM.
  Vd: public static void printIntegerList() {...}
  public static void printIntegerList(int n) {...}

III. Object & Class

1. Object

- Đối tượng là những “thứ-object-thing” quanh ta, hữu hình hoặc vô hình, chứa đựng nhiều thông tin giúp ta mô tả, nhận diện, phân biệt, đếm, chạm vào được.
- Object chứa nhiều thông tin được mô tả, nhận diện, phân biệt:
+ Tên gọi tắt/định danh (biến phức tạp/biến object).
+ Đặc điểm (biến và value).
+ Hành động/hành vi/behavior/method/hàm.

2. Class

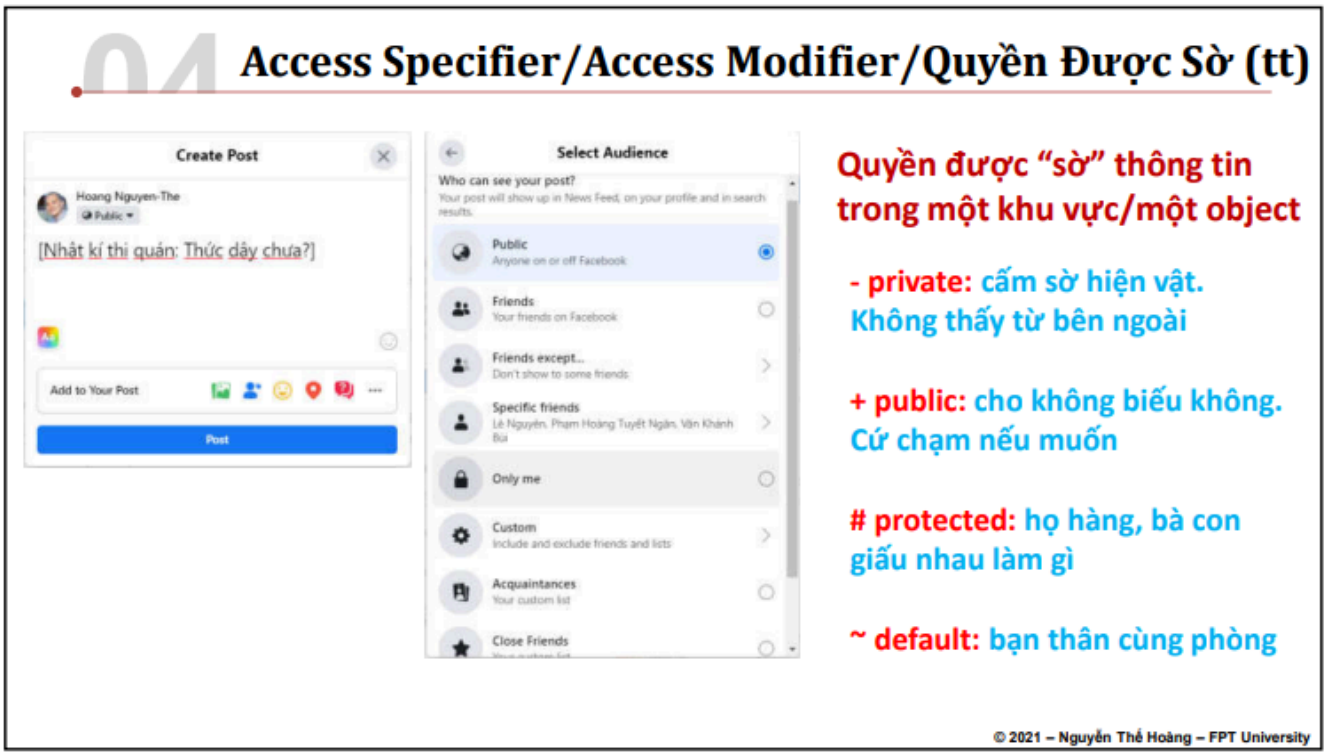
- Class là sự chia nhóm, phân loại các đối tượng tương đồng nhau về đặc điểm và hành vi.
- Class giống như một cái Khuôn chứa đựng nhiều thông tin chi tiết giúp ta mô tả một object.
  Vd: Khuôn đúc cúp World Cup chứa nhiều thông tin như cân nặng, chiều cao.
  CCCD là một Khuôn chứa đựng nhiều thông tin mô tả một công dân.
  Khuôn CaSi chứa đựng nhiều thông tin để mô tả các object đi hát.
- Khuôn là hình dáng của một object sẽ xuất hiện trong tương lai, gồm đặc điểm và hành vi (xử lý info - hàm).
- Khuôn phải kín, code phải viết trong class.
  Vd:
```
public class Dog {
        private String name;
        private int yob;
        private double weight;
        …
}
```

3. Constructor - Phễu

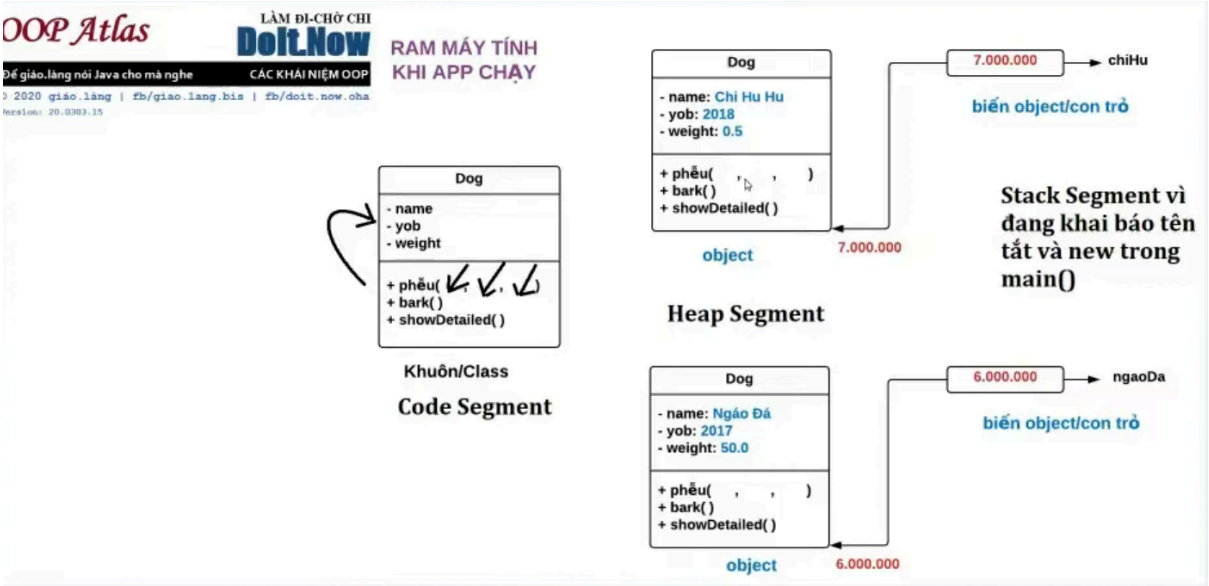
- Constructor/Phễu là một hàm rất đặc biệt, nó xử lý: ép dòng dữ liệu/vật liệu từ bên ngoài chảy vào đúng chỗ bên trong để xử lý - hàm (quan điểm của lập trình).
- Mỗi lần sử dụng phễu, gọi phễu tương đương với việc đúc 1 bức tượng mới, tạo 1 object mới.
- Tên phễu phải trùng 100% tên class, chữ hoa từng đầu từ.
- Hàm phễu không có giá trị trả về, không được có void, vì void sẽ nhầm lẫn với ứng xử/hành xử của object.
  Vd:
```
public Dog(String name, int yob, double weight) {
        this.name = name;
```

```
        this.yob = yob;
        this.weight = weight;
    }
```

4. Access Specifier / Access Modifier

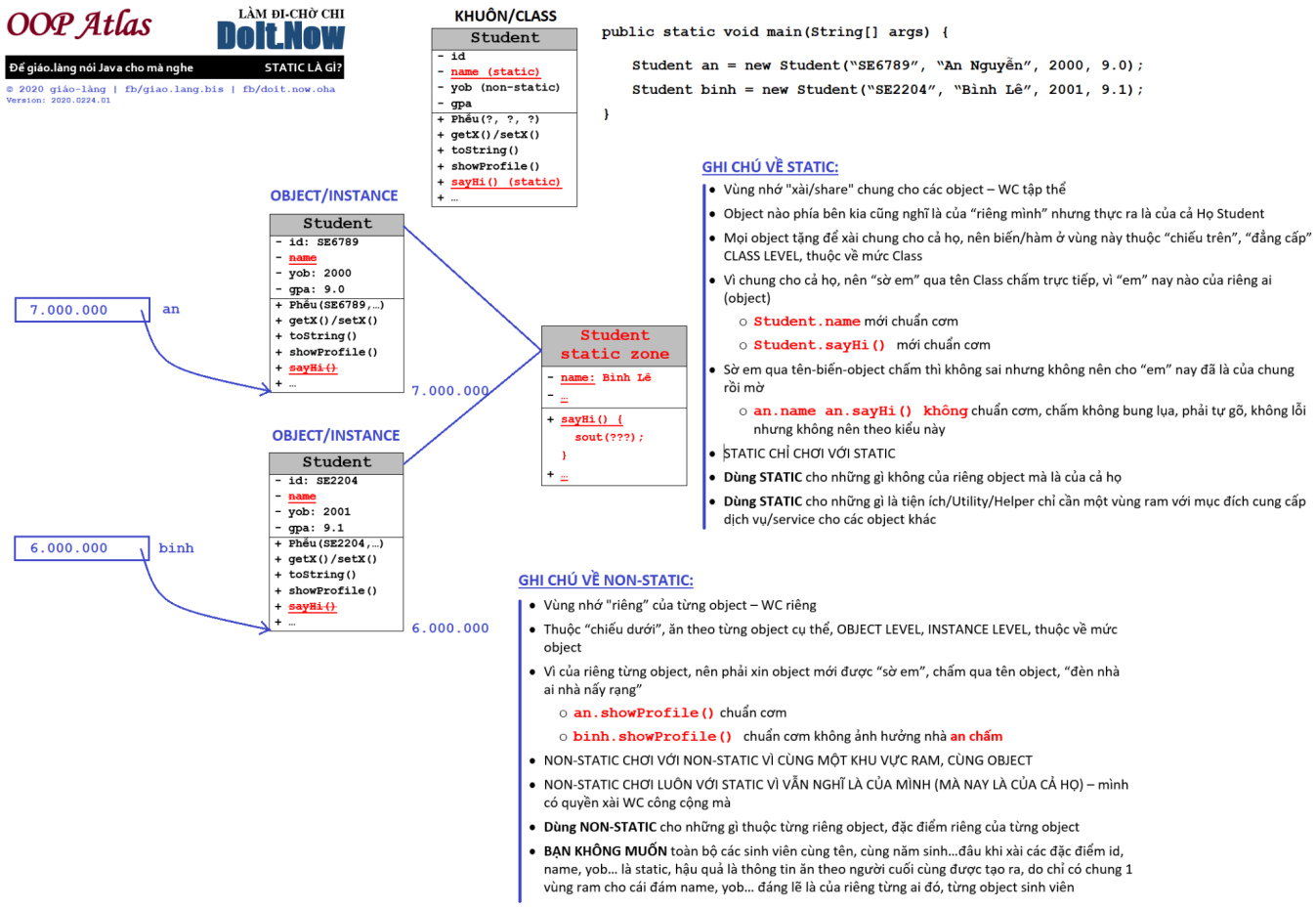


5. Object lưu trữ trong RAM như thế nào?



- Object tốn 2 vùng ram:
+ 1 vùng ram là vùng new - chính là object chứa đặc điểm và hành vi.
+ 1 vùng ram là biến con trỏ/biến object trỏ vào vùng new object. Chạm sờ vào object ta dùng biến object CHẤM. ON/OFF trong biến con trỏ/biến object, value của biến con trỏ là tọa độ/địa chỉ của vùng new.
- Biến nào khai báo trong hàm thì nằm trong vùng STACK SEGMENT.
- Object/vùng new nằm trong HEAP SEGMENT.

IV. Static



V. Input & Try-catch

- Để tránh trường hợp khi nhập dữ liệu từ bàn phím bị trôi lệnh, cụ thể là lệnh nhập số trước lệnh nhập chuỗi, ta có 2 phương án:

+ sc.nextLine() - xóa toàn bộ rác trong buffer trước khi thực hiện lệnh tiếp theo.
+ int yob = Integer.parseInt(sc.nextLine());

- Để tránh trường hợp người dùng vô tình hay cố ý nhập sai kiểu dữ liệu đầu vào mong muốn, ta dùng try-catch

```
Vd: while(true) {
        try {
            // Lệnh có khả năng xảy ra lỗi.
```

```
            int n = Integer.parseInt(sc.nextLine());
            break;
        }
        catch (Exception e) {
            // Hướng giải quyết hậu quả.
        }
    }
```

- Ta dùng try-catch khi:
+ Convert data không thành công (từ chuỗi thành số …).
+ Chia cho 0 nhưng không chặn số chia qua if.
+ Vượt số phần tử mảng (out of bound).
+ Null pointer - chưa gán giá trị cho biến con trỏ/biến object mà đòi chấm.
+ Kết nối mạng, truyền nhận dữ liệu qua mạng, database ở xa. Server chết, mạng lag.
+ Sql injection.
+ I/O nhập xuất với tập tin, bắt buộc phải nhốt lệnh vào trong try-catch.

  ….

VI. Array

1. Primitive array

- Mảng là kỹ thuật khai báo nhiều biến cùng lúc, cùng kiểu dữ liệu.
- Có 4 cách khai báo mảng trong Java:
+ Cách 1: int arr[] = {0, 5, 10, 15, 20, 25};
+ Cách 2: int[] arr = {0, 5, 10, 15, 20, 25};
+ Cách 3: int arr[] = new int[36];
- Cách 4: int arr[] = new int[]{0, 5, 10, 15, 20}; // explicit new
- Ngoặc [] là mảng, xin số biến.
- Ngoặc () là gọi phễu của Khuôn.

- Cứ có new là có một vùng ram lớn chứa 36 biến int với value mặc định là 0.
- Tên mảng là một biến lớp trưởng, con trỏ, reference trỏ vào đầu dãy các biến trong mảng. Tên mảng là biến lớp trưởng vì nó quản lý tất cả các bạn học trong lớp. Hỏi biến lớp trưởng ơi, trong lớp có bao nhiêu bạn, biến lớp trưởng chấm length - đặc điểm của biến lớp trưởng.
  Vd:

```
int arr[] = new int[36];
arr[0] = 39;
arr[1] = 79;

for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + " ");
}

for (int x : arr) {// Với mọi giá trị x thuộc mảng arr
    System.out.print(x + " ");
}
```

2. Object array

- Khai báo object array giống như primitive array:
  Student[] s = new Student[10];
  s[0] = new Student("SE123456", "AN NGUYỄN", 1998, 9.0);
  s[1] = new Student("SE123457", "BÌNH LÊ", 2000, 5.0);
- Các biến object còn lại trong mảng có giá trị mặc định là null, s[i] = null, tức là trỏ tới vùng đáy ram có địa chỉ là 0, chứa toàn số 0, byte null. Vậy nên khi duyệt for object array không nên duyệt hết mảng khi mảng chưa gán full giá trị cho tất cả biến trong mảng.

- Sorting an object array ascending by gpa, giả sử object array này đã gán full giá trị cho các biến trong mảng:

```
for (int i = 0; i < s.length - 1; i++) {
    for (int j = i + 1; j < s.length; j++) {
        if (s[i].getGpa() > s[j].getGpa()) {
            Student t = s[i];
            s[i] = s[j];
            s[j] = t;
        }
    }
}
```

VII. Inheritance

1. Định nghĩa

- Các mức nhận thức, thiết kế:
+ Object => Khuôn/Class là 1 mức nhận thức, thiết kế.
+ Khuôn A, Khuôn B, Khuôn C thấy chúng có điểm tương đồng:
- Đứng trên vai người khổng lồ: có sẵn rồi, làm thêm cho hay, làm ra cái mới, phát huy thêm, phiên bản mới từ cái cũ, ai đó dọn đường, ta đi tiếp.
- Chế tạo món mới từ món cũ, độ, điều chỉnh 1 chút ít, để ta có sản phẩm nhanh.
- Muốn có object ta cần Khuôn X, Khuôn X làm từ đầu thì mất công nên ta độ từ Khuôn Y nào đó. Khuôn X là bản điều chỉnh từ Khuôn Y, có nguồn gốc dính dáng từ Khuôn Y, dùng lại những ưu thế, thứ có sẵn từ Y.

  Vd: HV (Hình Vuông) IS A (KIND OF) HCN (Hình Chữ Nhật)
  HCN gọi tên là class CHA, parent, base, phiên bản gốc.
  HV gọi tên là class CON, child, sub, derived.
- KẾ THỪA không phải là copy/paste, kế thừa là xài lại 100% không cần gõ lại code.

- @Override là khi Con có hàm trùng tên 100% với hàm trong Cha thì Con qua mặt Cha, phủ quyết, vờ như không thấy hàm của Cha, đặc trưng cho 1 thứ gọi là tính đa hình, đa ánh xạ, đa nhân cách, 50 sắc thái, transformer, POLYMORPHISM.
- Lưu ý quan trọng: khi chơi với kế thừa phải tôn trọng phễu gốc của Con, chỉ cần độ lại 1 tí phễu của Con.

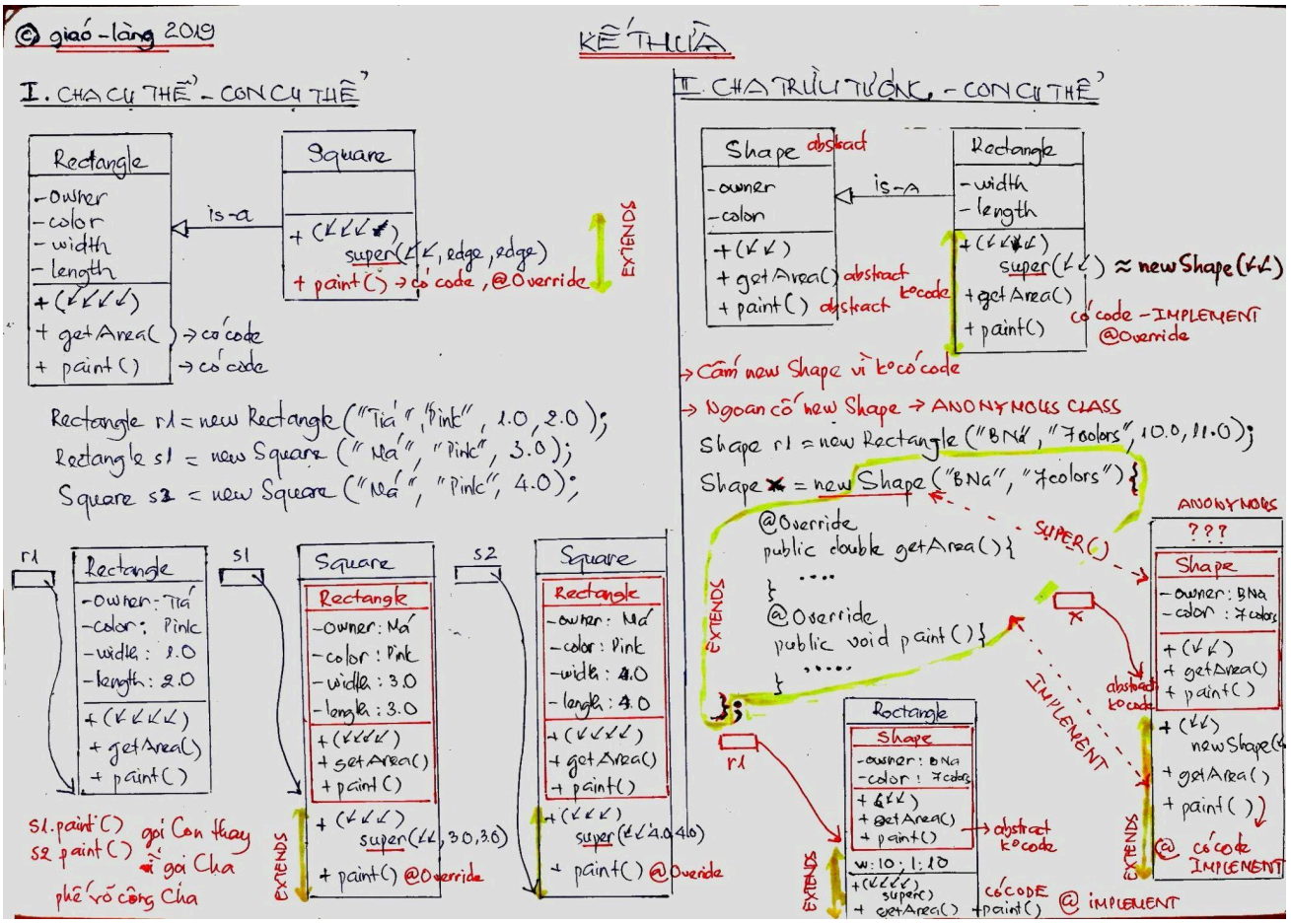
  Vd:

```
// Class Cha
public class Rectangle {
    protected String owner;
    protected String color;
    protected double width;
    protected double length;

    public Rectangle(String owner, String color, double width, double
length) {
        this.owner = owner;
        this.color = color;
        this.width = width;
        this.length = length;
    }
    …
    public void paint() {
        System.out.println("RECTANGLE …");
    }
}

// Class Con
public class Square extends Rectangle {

    public Square (String owner, String color, double edge) {
        super(owner, color, edge, edge); // ~~~ new Rectangle(...);
    }
```

```
        @Override
        public void paint() {
                System.out.println("SQUARE …");
        }
}
```

2. Class Con, Class Cha lưu trữ trong RAM như thế nào?

- Liên quan đến kế thừa, có 2 điều:
+ Di truyền: cha có gì con xài nấy, cha thêm, con có thêm, cha bớt, con bị bớt.
+ Biến dị: con khác cha à nhen, điều gì xảy ra - kỹ thuật DRIFT, rê bánh.



- Vùng nhớ Con to/bao trùm vùng nhớ Cha với ý nghĩa:
+ Con thừa hưởng tất cả đồ của Cha (trong vùng new có gì thì được xài cái đó).
+ Cha là nền tảng/móng nhà/base để Con thoải mái vùng vẫy, vùng nhớ Cha mở rộng/extends thêm 1 chút thành vùng nhớ Con.
+ Cha sẽ là cánh chim, đưa Con bay thật xa. Ngôi nhà cũ, cơi nới thêm thành nhà mới.
+ Vùng nhớ Con = vùng nhớ Cha + extends (mở rộng thêm)
+ super()-di truyền| @override, điều khác-biến dị

- CHỐT CON TRỎ:
+ Khai Cha = new Cha. Chấm xổ ra tất cả của Cha.
+ Khai Con = new Con. Chấm xổ ra tất cả của Con:
● xổ tất cả của Cha (thừa kế) - di truyền.
● xổ tất cả phần @Override và của riêng Con (extends) - biến dị.
● Con thêm hàm gì đó, không phải @Override thì Cha không biết.

+ Khai Cha = new Con. Chấm xổ ra:
● Xổ tất cả của Cha (thừa kế - di truyền).
● Không xổ @Override vì trùng tên hàm.
● Không xổ ra những gì của riêng Con, chấp việc new Con, chấp việc trong ram đã có biến dị. Lí do là vì đang dùng con trỏ Cha, thì chỉ xổ kiểu Cha, con trỏ chỉ trỏ đến vùng super (Cha nằm trong Con, không chịu nhìn xuống dưới extends).
● Để thấy được hàm dị biệt của Con, phải DRIFT con trỏ, kéo giãn sợi dây thun, ép kiểu, 2 cách nhưng bản chất là 1, kéo xuống đáy vùng new, chụp lại/giữ lại tọa độ new Con, muốn chấm gì thì chấm.
  Parent c = new Child(“NHÀ PHỐ”, “100 ETH”);
  Cách 1: Child x = (Child) c; x.sayHello();
  Cách 2: ((Child)c).sayHello();

3. Abstract class

- Ta cần chế tạo nhiều Khuôn mà lại CHẾ TẠO nhanh để từ đó ra object cho nhanh => vẫn là kế thừa mà thôi, vẫn dùng lại Khuôn nào đó, vấn đề là dùng lại bao nhiêu Khuôn để làm nhiều Khuôn?

+ Mục tiêu đa dạng object nhưng cần nhanh mà lại chung phiên bản, tương đồng nhau.
+ Ta cần 1 KHUÔN TỔ, từ đó "đẻ" ra được nhiều Khuôn Con "khác nhau", đám khác nhau về chung 1 nhà, chung mảng, sắp xếp sẽ yên vui.

  Vd: Vuông, Tròn, Chữ Nhật chung 1 nhà!!!

  Vuông, Tròn, Chữ Nhật IS A KIND OF …?

  Vuông là Chữ Nhật, chế tạo Vuông rất nhanh

  Tam Giác Vuông là Tam Giác, chế tạo TGV nhanh

  Vuông, Tròn, Chữ Nhật, Tam Giác, Hình Bình Hành, Oval… is a kind of SHAPE (HÌNH HỌC), phàm là Hình Học sẽ có đường nét, màu sắc, diện tích, chu vi. Shape: Khuôn Tổ, từ đây ra cả đám Khuôn mới.

- Cái gì tồn tại nhưng không chạm nắm được, chỉ mang ý nghĩa khái niệm, nói về 1 ý tưởng, cái đó hữu nhưng mà vô, có thấy nhưng không chạm được - ABSTRACT - TRỪU TƯỢNG.
- S (diện tích), P (chu vi) của Shape là trừu tượng, tồn tại để nói về thứ cần đo đạc, chưa cụ thể vì ngay cả Shape cũng là 1 khái niệm chưa cụ thể, biết hình gì đâu? Vẽ cho mình 1 cái hình, hỏi lại là hình gì? Chúng là những khái niệm sẽ được nạp vào đầu mình để biết rằng sẽ có gì đó diễn ra tiếp theo: KHÁI NIỆM - ABSTRACT - TÍNH TRỪU TƯỢNG CHÍNH LÀ NÓI VỀ 1 Ý TƯỞNG.
- Một class chứa hàm abstract tức là hàm không code, tức là ý tưởng cần có, thì bản thân class chưa hoàn hảo, chưa hành xử

được, giống như bản concept, nó chỉ mới là ý tưởng mà thôi, do đó nếu class mà chứa hàm abstract thì class cũng phải là abstract.

```
public abstract class Shape {
    protected String owner;
    protected String color;
    …
    public abstract double getArea();
    public abstract double getPerimeter();
    public abstract void paint();
}
```

4. Các bước thực hiện Inheritance

- Bước 1: tìm mối quan hệ IS A giữa Cha và Con. Con IS A KIND OF CHA.
- Bước 2: Đặc điểm:
+ Nếu Con trùng 100% Cha, gửi Cha giữ lấy không cần làm gì cả.
+ Con có dị biệt thì làm như bình thường.
- Bước 3: Phễu Con, thỏa hiệp nếu cần, có thể thay đổi thứ tự arguments cho thuận chiều.
- Bước 4: Hàm của Con:
+ Cha cụ thể, Con cụ thể (Concrete):
● Class Con chỉ cần làm phễu là xong, xài 100% từ Cha, thêm @Override và phần dị biệt nếu cần, muốn làm gì thì làm.
+ Class Con thừa kế Cha là abstract, Con chỉ có 2 lựa chọn:
● Con làm biếng không thèm làm những điều Cha nói, không thèm viết code cho các hàm abstract từ Cha, tức là lúc này Class Con chứa hàm abstract từ Cha, mà class chứa hàm abstract tức là class đó chỉ là khái niệm, chưa hoàn hảo. Một class là abstract tức là khái niệm, giống như bản concept, không new để tạo object được. Cha không thể NEW vì Cha chưa hoàn hảo, Con cũng thế thì dòng họ vô sinh vì không NEW tạo object được.

- Con có trách nhiệm thực hiện những điều Cha nói, viết code cho các hàm abstract từ Cha, vì nếu không làm thì sẽ vô sinh do chưa hoàn hảo. Con nếu không muốn vô sinh thì phải viết code cho các hàm abstract từ Cha, implement all of the parent's abstract method, @Override.

5. Anonymous Class

- Liên quan tới hình học, có xuất hiện những đối tượng lạc loài, không hình dạng quen thuộc, là hình cắt ngẫu nhiên, không biết sắp xếp vào nhóm nào ngoại trừ nhóm Cha là Shape.
- Theo quan điểm object thì đã có object được tạo ra, có 1 hình mà, mà không biết Khuôn là gì, chỉ biết nó là thuộc Khuôn Cha Shape.
- Dùng kĩ thuật mượn gió bẻ măng, mượn Shape tạo object, vì ta thiếu 1 cái Khuôn con cụ thể, nếu có Khuôn cụ thể (full code) new ngay Khuôn này ra hình mình cần.
- Con = new Cha + code @Override vẫn đảm bảo công thức vùng RAM Con như xưa nay, đủ code tạo object mà không cần class riêng, mượn Cha mà làm, class riêng mà không cần đề cập, gọi là class làm biếng, class không thèm làm cụ thể ra gọi là class ẩn danh, anonymous.
- Vẫn new được object con của Shape mà không cần 1 class con cụ thể, vậy thì vùng ram con này chính là đại diện cho 1 class làm biếng làm riêng/đặt tên, class ẩn danh.
- Tạo object on the go, làm biếng tạo class con có sẵn, gọi là ANONYMOUS CLASS. Không chỉ định rõ tên class con mà vẫn có object con.
- Khai Cha new Cha, nếu Cha abstract, mượn gió bẻ măng, Cha new đỡ cho Con, viết code on the go, đừng quên chấm phẩy
  Vd: Shape vangLai = new Shape (owner, color, borderColor) {
  @Override
  public double getArea() {return 50; // giả sử S là 50}

```
        @Override
        public double getPerimeter() {return 40; // giả sử P là 40}

        @Override
        public void paint() { … }
};
```

VIII. Interface

- Có 2 hình thức gom nhóm:
+ Gom trên đặc điểm và hành vi - INHERITANCE (họ hàng bà con). Inheritance/abstract class: tên của 1 dòng họ nhiều con cháu chia sẻ đặc điểm và hành vi. IS - A - KIND - OF.
+ Gom trên hành vi, tụ tập nhau làm gì đó - INTERFACE - chung giao tiếp (hội nhóm). Interface: tên của 1 hội nhóm ăn chơi hành động. IS - A - MEMBER - OF.
  Vd: Bạn là Worker, Employee, Student, Lecturer, bạn cần cải thiện tiếng Anh, bạn tham gia vào 1 CLB-Tiếng-Anh, bạn được đòi hỏi rằng chỉ cần speakEnglish là đủ, không quan tâm bạn xuất thân từ đâu, chúng ta chơi chung, giao tiếp bằng tiếng Anh. Worker, Employee, Student, Lecturer IS A MEMBER OF ENGLISH-CLUB, method duy nhất: speakEnglish().
- CLB chỉ quan tâm hành động, trong đây chỉ chứa hành động. Interface/CLB để mọi người tự chơi theo cách của mình - hành động là abstract. Chỉ nói nội quy CLB thôi heng, mọi người theo đó mà làm. Interface chứa hàm abstract, không cần ghi ra từ khóa abstract ở cả tên CLB và tên hàm vì CLB hàm ý/ngầm hiểu chỉ có nội quy để mọi người theo - implement - ngầm hiểu Interface là Abstract - nêu ý tưởng. Tham gia CLB nào thì phải có hành động của CLB đó, implements all abstract methods bên CLB.
- Trong Java, 1 Con tối đa 1 Cha, đơn kế thừa. 1 Con tham gia N CLB, vậy sẽ có hành động của N CLB.

- Code:

```
public interface DeathRacer {
    public double runToDead();
    public void showHowToDead():
}

public class Con/Hội-Viên implements CLB1, CLB2, CLB3 {
    @Override tất cả các hành động của các CLB;
}
```

+ CLB không có đặc điểm, không có phễu/constructor - nhưng có hàm abstract. Khi new Con thì CLB vẫn sẽ được new để “Cha” sẽ làm cánh chim, đưa Con extends @Override
+ Member/Con vẫn có 1 vùng RAM của CLB - chứa hàm abstract, con trỏ vẫn trỏ lưng chừng CLB.
+ Vẫn theo mô hình RAM của abstract class/Cha truyền thống: Con to hơn Cha/CLB về vùng RAM, super() Cha CLB là super() ngầm.

```
public class Con/Hội-Viên extends Cha implements CLB1, CLB2, CLB3 {
    @Override tất cả hành động của Cha nếu Cha là abstract class;
    @Override tất cả các hành động của các CLB;
}
```

+ Con đóng N vai: vai dòng họ, vai CLB.
+ Nếu Cha, CLB có cùng tên hàm f() ngoài những hàm khác, thì Con có bị bối rối không? Con không bối rối, vì f() của Cha và các CLB toàn là abstract, Con phải code @Override duy nhất hàm f(), không sợ ambiguous.

- ANONYMOUS:

+ Object vãng lai tham gia CLB, vậy thì anh cũng phải hành xử như việc anh tham gia thường xuyên, phải có hành động tương ứng mới được tham gia, chỉ ngay lúc đó thôi, ON-THE-GO.
+ Ghé ngang mà chơi, thì phải chơi/hành xử như người ta đang chơi.
+ Bản chất ta có 1 nhóm nào đó trước đó, nhưng ta chỉ cần hòa nhập/chơi/tham gia lúc này, không cần bộc lộ rõ trước đó ta là ai, nhóm nào, chỉ cần biết join nhóm mới lúc này.
  Vd: CLBCờ x = new Student(...); // một SV tham gia CLB Cờ - code sẵn @Override.
  // gã ghé ngang, không chuyên nhưng ham vui
  CLBCờ hangXom = new CLBCờ() {
  @Override
  Anh ham chơi, ghé ngang tham gia, thì phải có hành động tương ứng;
  }

+ ABSTRACT CLASS và INTERFACE đều là 2 thứ không hoàn hảo, chỉ nói mà không triển khai làm. Con/Member cần phải hoàn tất nốt câu chuyện không hoàn hảo - @Override.
+ Object vãng lai, không xác định nhóm trước mà muốn tham gia cuộc chơi, dòng họ, thì phải hành xử/cư xử như dòng họ, member.
+ Không gọi chính xác nhóm của bạn, nhưng gọi bạn theo nhóm bạn tham gia, theo nhóm bạn chơi được mà. Vùng RAM và mọi hành xử chấm, @Override qua mặt như bình thường, do làm biếng tạo class rời - concrete class.

- Ứng dụng của Interface:
+ Giúp 1 class/object có nhiều cách hành xử khác nhau, tôi là member của Hội nào thì tôi có hành động của Hội đó. Giúp cho 1 object đóng nhiều vai, vai nào thì có hành động tương ứng. Chính là 1 “hình dạng” của đa kế thừa - multi-inheritance.

+ Loose coupling (gắn kết lỏng lẻo) giúp cho cách tổ chức các class có ràng buộc nhau để làm nên câu chuyện lớn nhưng lại dễ dàng mở rộng và thay đổi.
+ Dùng Interface là cách viết code giúp linh hoạt, dễ dàng thay đổi và mở rộng code mà không phải sửa phần code cũ, thích ứng cho tương lai.
+ Interface là cách kết giao, mở rộng giao tiếp kết nối với xung quanh.
+ Comparable/Comparator => dính đến Loose Coupling.
+ Mục tiêu cuối cùng của Interface: chơi chung với nhau!!! Dù bạn khác biệt trong hành xử. Mỗi bạn đánh cờ khác nhau, nhưng chung ván cờ. Mỗi CARD đồ họa khác nhau về năng lực xử lý, nhưng chung khe PCI-EX.

IX. OOP Concepts / OOP Principles

1. Abstraction - trừu tượng hóa mọi vật

- Là cách tư duy, cách suy nghĩ, cách biểu diễn những thứ phức tạp trong cuộc sống thành những thứ đơn giản hơn nhưng vẫn diễn tả đúng ý nghĩa của sự vật ban đầu.
- Cách lấy ra những thông tin quan trọng nhất, tiêu biểu nhất của một object phức tạp ngoài đời để biểu diễn lại cho ai cũng hiểu được thứ phức tạp này .
- Một thứ phức tạp ngoài đời được biểu diễn qua một thứ đơn giản hơn, vẫn không làm mất chất.
- Using simple things to represent the complex things. Extract the significant features/characteristics of a complex thing to form a simple/easy catching thing without losing the original meaning of the complex one.
- Tư duy nắm bắt thông tin cốt lõi quan trọng của object ngoài đời, dùng để biểu diễn lại object đó.

Vd: Sinh viên ngoài đời rất phức tạp về thông tin: tên, tuổi, địa chỉ thường trú, tạm trú, lịch trình di chuyển, gia cảnh, sở thích, chuyên ngành học, ….

+ Trường đại học không cần nắm hết thông tin này, vẫn có thể quản lý được object phức tạp này. Tìm ra những thông tin đặc trưng đủ để quản lý, biểu diễn, minh họa một sinh viên.
+ Sinh Viên (id, email trường, tên, địa chỉ thường trú, phone, chuyên ngành, điểm các môn), rút gọn/extract chỉ lấy những thông tin quan trọng nhất, có ý nghĩa nhất ra để minh họa một sinh viên bất kỳ.

⇒ Tìm đặc điểm quan trọng của một object, tìm đặc điểm cho một class/khuôn => ABSTRACTION. Biến đổi/biểu diễn thông tin không khiến mất gốc ban đầu. Mô tả lại một câu chuyện/thứ nào đó để ai cũng hiểu được - tính trừu tượng/tính mô tả/ngắn gọn. Trừu tượng hóa tức là kể lại câu chuyện mô tả trong trí não/hình dung trong đầu kể lại cho ai đó nghe và người ta hiểu được.

2. Encapsulation - cô lập đóng gói trọn vẹn đúng thứ cần làm

- Nhận diện 1 đối tượng gồm đặc tính (biến - data) và hành vi (hàm - xử lý data), gom chung trong 1 đơn vị object/class.
- Gom xong giấu/che đi bằng thuật ngữ private, show ra nếu muốn theo cách public/protected. Tạo class đóng gói các đặc điểm + hành vi, che và không che.
- Vd:
+ Giống y chang viên thuốc con nhộng - vỏ ngoài là protein kiểu nhựa sinh học, tan khi uống vào, tách viên thuốc bên trong thấy bột thuốc. Nhìn từ bên ngoài vỏ thuốc con nhộng, không thấy thuốc bên trong, ENCAPSULATE (v) gói lại che giấu, có thể trong suốt nếu muốn, nhưng trong con nhộng chắc chắn chỉ có thuốc.

+ Tivi: che bằng cái vỏ nhựa/màn hình, ta chỉ tương tác qua remote/nút bấm. Không thấy bên trong tivi ra sao như xử lý thông tin + quét data bộ phim qua wifi/vô tuyến. Đứng ngoài tivi giao tiếp qua public nút bấm. Tivi chính là ENCAPSULATION.
- Encapsulation chính là cách gom biến/data và hàm vào 1 đơn vị object/class. Chỉ làm đúng việc của nó được chỉ định thiết kế. Gom gì thì cần ABSTRACTION để lấy cái đặc trưng. Tivi xem phim, không gánh thêm việc lò nướng.
- ABSTRACTION (nhận diện object) là lấy ra đặc trưng, ENCAPSULATION (tạo class instance variable + method()) là đóng gói trọn vẹn đúng thứ cần làm.

3. Inheritance - tái sử dụng, có xài luôn

- Reuse lại code mà không cần phải làm lại code, thậm chí không cần phải copy & paste.
- Tăng tốc quá trình viết code khi có đoạn code, object đã làm tốt việc của nó rồi, ta tận dụng để mở rộng thêm khả năng của object mới, dùng lại đồ object cũ + dị biệt ta làm thêm.
- Tăng tốc làm: ba má giàu có, mình thuận lợi trong việc có vốn làm ăn. Ba má cho căn nhà, mình đỡ lo tiền trọ, ta chỉ lo tiền ăn. Thay vì bòn rút kiểu bất hiếu - xài mà không chịu làm thêm, nay ta bật nhanh hơn, hội nhập nhanh hơn, tham gia vào cuộc sống nhanh và tự tin hơn.
- Có thư viện viết code làm app cho nhanh, không làm lại từ đầu. Có thư viện, ta mở rộng thư viện cho nhu cầu riêng, để sau này đa năng sử dụng, tăng tốc viết code.
- Cha cụ thể, Cha trừu tượng, Abstract Class chẳng qua là cách gom, phân loại đối tượng mức cao thấp, giống như phân loại gom nhóm ngoài đời. Tận dụng kế thừa thành quả object có cái gì? Dùng lại nguyên tắc ABSTRACTION + ENCAPSULATION.

4. Polymorphism - đa hình/đa ánh xạ - từ một ra nhiều

- Mùa đông 1946, Bác Hồ kêu gọi toàn quốc kháng chiến, ai có súng dùng súng, ai có dao dùng dao….
+ Chỉ là 1 câu kêu gọi: chiến().
+ Nhân dân quật khởi đứng lên: dao, kiếm, liềm, súng,... đa dạng cách giết giặc, đa dạng cách implements hành động chiến().
+ Từ 1 chữ chiến() có N cách làm khác nhau, tính đa dạng, biến hình, đa sắc thái, đa nhân cách, đa hình, POLYMORPHISM.
- Đại ca kêu tối nay giang hồ chợ lớn, đi chiến nha. Đàn em đem hàng: mã tấu, dao, côn nhị khúc, ….
- Các Vua Hùng đã có công dựng nước, Bác Cháu Ta phải implements/@Override. Từ 1 hàm Cha, có N Con @Override qua mặt, chính là tính đa hình.
- Gọi Cha mà Con chạy, đảm bảo tính đa dạng của các hành động, vẫn chơi chung và thống nhất hành động qua tên gọi.
- @Override hiện thực cho tính đa hình.

X. Collections Framework

1. Tủ loại 1: Mảng [Số đồ để vô]

- Fix cứng số đồ để vô, chỉ để bỏ vô 1 số giới hạn món đồ.
- Mảng là khai báo nhiều biến nhiều chỗ chứa [fix số chỗ].
- Mảng y chang xe ô tô chở khách [16 chỗ], rạp phim A [100 ghế].
- [i] = new món đồ() cứ thoải mái nằm đâu đó, ta lưu tọa độ.
- Vd: Student arr[] = new Student[100];

2. Tủ loại 2

- Dù tủ loại 1 hay 2 thì đều chứa nhiều đồ: đồ để thật sự vào trong tủ hay chỉ giữ danh sách tham chiếu đều được. Tủ hồ sơ giữ danh sách tham chiếu hay tủ quần áo chứa đồ thật luôn - mảng primitive value lấy xài luôn.
- Co giãn số đồ để vô, thoải mái để đồ vô, cần thêm thì có thêm chỗ, không cần xin, giống nồi cơm thạch sanh. Ban đầu không có chỗ nào cả, không đáy, cứ nhét đồ vô thì thêm chỗ.
- Không sờ [i] vào vị trí chỗ, xin qua dây kéo, khóa, mở ra nhét đồ vô: tuCuaTui.add(món đồ bỏ vô) - bỏ đồ vô qua hành động của tủ, không sờ trực tiếp ngăn chứa [i], bỏ vô tủ tự lo bên trong.
- NEW tủ này, giống các thương hiệu túi xách balo bạn mua, rồi ADD(món đồ khác đi mua). Đều chứa tham chiếu, linh hoạt trong việc để các món đồ, bệnh nhâu đâu cần dồn vào phòng bác sĩ, bác sĩ vẫn điều trị được vì có danh sách bệnh nhân trong tủ hồ sơ.
- Phân loại nâng cao cho tủ cụ thể, class Cha - abstract:

List	Set	Map
Danh sách	Tập hợp	Bộ ánh xạ, từ này ra kia: KEY, MÓN ĐỒ - VALUE
Vào thứ tự nào ra thứ tự nấy, mảng nâng cao co giãn kích thước. Vào trước, ngồi [0], vào sau, ngồi [1]	HashSet lộn xộn thứ tự. TreeSet sắp xếp tự động. Không được lặp lại món đồ lần thứ 2.	
Thống kê số vé bán, số lượt người mua, một người có thể đếm nhiều lần.	Đếm số người thực sự xem phim, không đếm lượt, mỗi người xuất hiện 1 lần,	

	không hơn dù bạn xem nhiều lần.	

- Các tủ thật túi thật mua về để xài, new được, không phải là abstract, hoàn hảo để dùng, full code hành động, class Con:

ArrayList, Vector	HashSet, TreeSet	HashMap, TreeMap
- List<Student> arr = new ArrayList<>(); arr.size(); trả về kích thước mảng. - arr.add(new Student(...)); - arr.get(i); trả về tọa độ con trỏ/món đồ thứ i. - arr.set(i, biến object - tọa độ vùng new);	- Set<Student> arr = new HashSet<>(); - Không có hàm get(), muốn lấy thông tin trong Set chỉ có cách FOR EACH.	

XI. Wrapper Class

1. Tổng quan

- Ta có các số: 1, 2, 3, 5, 7, -5, -10 - int

+ Mô tả/đặc điểm:
  - Giá trị/đại lượng: 1, 2
  - Nguyên con, trọn vẹn không lẻ miếng nào về giá trị.
+ Mô tả hành động:
  - 1 < 2

- 3 > 0
- So sánh các đại lượng của nhau

+ Hóa ra các đám ở trên có chung đặc điểm là không lẻ miếng nào, mang giá trị, còn so sánh được lẫn nhau => chúng thuộc nhóm Integer.

```
public class Integer {
    private int value;

    public Integer(int value) {
        this.value = value;
    }

    // get()

    // HÀM SIÊU ĐẶC BIỆT
    public static int parseInt(String num) {
        biến chuỗi thành số để đem xài;
    }
}
Integer year = new Integer(2026); // không khuyến khích
Integer year = 2026; // new ngầm (lõi là 2026)
```

- Cơ chế boxing/đóng hộp/wrapping: khi ta gói cái primitive => object.
  Vd: int n1 = 2003;
  Integer n2 = n1;
- Unboxing: khi ta mở hộp, lấy value lõi ra gán cho biến primitive vẫn tương thích, phù hợp. Unboxing: object => primitive.
  Vd: int n3 = n2;
- Wrapper Class không có khái niệm set(), tức là mỗi lần gán value là 1 lần new mới, không xài vùng new cũ, không cho phép thay đổi value lõi, tức là read-only.

- Một class bất kỳ mà chỉ có hàm get() không có hàm set(), không có hàm cho thay đổi bên trong ngoại trừ lúc đúc, class read-only, thì class này được gọi Immutable Class. Class nào mà cho phép thay đổi value đã được đúc gọi là Mutable Class.
- Có tổng cộng 8 Wrapper Class ứng với 8 kiểu primitive.

Byte	Short	Integer	Long	Float	Double	Boolean	Character
byte	short	int	long	float	double	boolean	char

2. Pool

- Pool là 1 vùng ram chứa các vùng new của Wrapper (số nguyên) và String, giống như hồ bơi ngoài đời, nhiều object tắm chung, chỉ áp dụng kĩ thuật hồ bơi trong một số tình huống đặc biệt, không nhầm với static.

+ Wrapper (Integer, Long): chơi new ngầm/boxing new ngầm trong đoạn [-128, 127] thì xài pool, 2 3 chàng trỏ 1 nàng, còn nếu new trực tiếp gõ ra thì là 2 vùng ram khác nhau.
  Integer n1 = 127; // new ngầm
  Integer n2 = 127;
  n1 == n2 => true

● Chốt hạ: cấm tuyệt đối so sánh 2 biến object qua toán tử == vì đi so sánh 2 địa chỉ, lúc bằng lúc không do có Pool hay không, chính xác phải so sánh 2 object qua dấu chấm vào trong để lấy data mà so sánh. Chấm để so sánh object!

+ Chuỗi new ngầm là Pool.
  String s1 = “HELLO”;
  String s2 = “HELLO”;
  String s3 = “Hello”;
  s1 == s2 => true
  s1 == s3 => false, Pool phân biệt hoa thường.

- Chốt hạ: không so sánh 2 chuỗi qua toán tử == vì lúc đúng lúc sai do chung pool hay không, so sánh 2 biến object là vô nghĩa, bắt buộc vào vùng new mà so sánh, CHẤM để mà so sánh: .compareTo(); .equals();

- Mọi object mình tạo ra hay người ta tạo sẵn ra thì luôn nên có 2 hàm ứng với 2 loại câu hỏi:

+ Tôi bằng bạn không? Tôi.equals(Bạn) => true hoặc false - boolean.
+ Tôi so với bạn như thế nào? Tôi.compareTo(Bạn) => -1 (nhỏ hơn), 0 (bằng), 1 (lớn hơn) - int.